Lecture 02: Node.js Module Systems

`Module System` code ko organize karne ka dhang hai — bina iske aapka codebase ek bheed ban jaayega. Is lecture mein hum Node.js ke module systems ko ground level se samajhenge — `CommonJS` aur `ES Modules` dono.

> INTUITION
>
> **HTTP ek contract hai.** Jaise restaurant mein menu card ek contract hai — "tum order do, hum khana denge" — waise hi HTTP ek written contract hai jo define karta hai ki client aur server ke beech request aur response kaise bhejne hain. Agar contract todoge, communication fail hoga.

URL Parsing & Query Strings in Node.js

Why URL Parsing Matters

Jab user `http://localhost:3000/30` hit karta hai, toh server ko ye number extract karna padta hai. Agar aap manually loop se / remove kar rahe ho, toh aap JavaScript ke built-in power waste kar rahe ho.

> **Problem 1: Manual URL Parsing**
>
> Loop se character remove karna instead of `slice(1)` — verbose aur error-prone hai.

```
// ❌ Manual way — verbose
const str = request.url;
let str1 = "";
for(let i = 1; i < str.length; i++)
    str1 += str[i];
let number = Number(str1);

// ✅ Clean way — one line
const number = Number(request.url.slice(1));
```

> **Problem 2: Invalid URL Handling**
>
> User `/abc` hit kare toh `Number("abc")` = `NaN`. Loop `NaN` times chalega, empty array return hoga. User confuse hoga.

```
if(isNaN(number)) {
    response.writeHead(400, { 'Content-Type': 'application/json' });
    response.end(JSON.stringify({ error: "Please provide a valid number" }));
    return;
}
```

Problem 3: Out of Range Request

User `/500` maange jab sirf 100 profiles hain. Array se bahar loop chalega, `undefined` values return hongi.

```
if(number > gitHub.length) {
    response.writeHead(400, { 'Content-Type': 'application/json' });
    response.end(JSON.stringify({
        error: `Maximum ${gitHub.length} profiles available`
    }));
    return;
}
```

BEST PRACTICE

Har incoming request ke liye 3 questions zaroor poochho:

1. **Input valid hai?** → `isNaN` check karo
2. **Input range mein hai?** → array bounds check karo
3. **Response ka format set hai?** → `Content-Type` header hamesha set karo

Ye 3 checks aapke 90% bugs ko rok degi.

Query Strings

WHY query strings? Kyunki GET request mein body nahi hoti. Data bhejna hai toh URL mein encode karna padta hai. Jaise search filters, pagination, sorting parameters.

```
http://localhost:3000/add?num1=10&num2=20
//      path    →   /add
//      query   →   ?num1=10&num2=20
```

? path end aur query start ko mark karta hai. & multiple values ko separate karta hai.

NODE.JS MEIN QUERY STRING PARSE KARNA

```
const http = require('http');
const url = require('url');

const server = http.createServer((req, res) => {
    const parsed = url.parse(req.url, true);

    const operation = parsed.pathname.slice(1);  // 'add'
    const num1 = Number(parsed.query.num1);      // 10
    const num2 = Number(parsed.query.num2);      // 20

    res.writeHead(200, { 'Content-Type': 'application/json' });

    if(operation === 'add') {
        res.end(JSON.stringify({ result: num1 + num2 }));
    }
    else if(operation === 'sub') {
        res.end(JSON.stringify({ result: num1 - num2 }));
    }
    else if(operation === 'mult') {
        res.end(JSON.stringify({ result: num1 * num2 }));
    }
    else if(operation === 'divide') {
        res.end(JSON.stringify({ result: num1 / num2 }));
    }
});

server.listen(3000);
```

URL.PARSE() KYA RETURN KARTA HAI

```
url.parse("/add?num1=10&num2=20", true)

// Returns:
{
    pathname: '/add',           // path part
    query: {
        num1: '10',             // query params as object
        num2: '20'
    }
}
```

IMPORTANT

`url.parse()` ka second argument `true` bahut important hai. Ye query string ko automatically JavaScript object mein convert karta hai. Bina `true` ke, query ek raw string `"num1=10&num2=20"` rahegi — jo manually parse karni padegi.

CommonJS Module System — require &

exports

What is a Module?

WHY modules? Kyunki ek file mein 5000 lines ka code likhne se **maintainability** khatam ho jaati hai. Modules code ko chote-chote files mein tod dete hain — har file ka ek specific responsibility hoti hai.

```
user.js        →   sirf user related code
payment.js     →   sirf payment related code
email.js       →   sirf email related code
```

module.exports

Node.js har file ke liye ek `module` object banata hai. Uske andar `exports` property hoti hai jo empty object `{}` se start hoti hai. Jo bhi aap `module.exports` mein daalte ho, wo hi dusri file mein `require()` se milta hai.

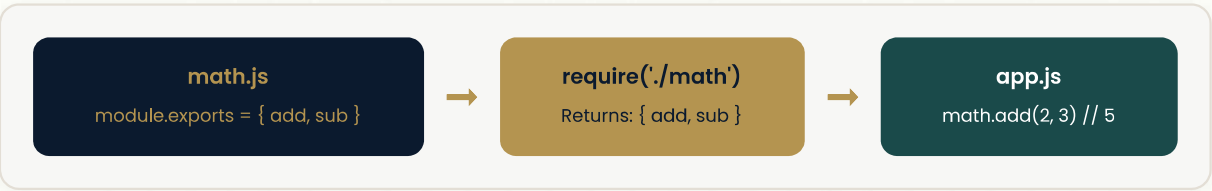

Figure 4: CommonJS Module Flow — Only module.exports crosses the boundary

EXPORT STYLES

1. SINGLE FUNCTION EXPORT

```
// math.js
function add(a, b) {
    return a + b;
}

module.exports = add;
```

```
// app.js
const add = require('./math');
add(2, 3);    // 5
```

2. MULTIPLE FUNCTIONS AS OBJECT

```
// math.js
function add(a, b) { return a + b; }
function subtract(a, b) { return a - b; }
function multiply(a, b) { return a * b; }

module.exports = { add, subtract, multiply };
```

```
// app.js
const math = require('./math');
math.add(2, 3);          // 5
math.subtract(5, 2);     // 3
math.multiply(2, 3);     // 6
```

3. DESTRUCTURED IMPORT

```
// app.js — sirf jo chahiye wo import karo
const { add } = require('./math');
add(2, 3);    // 5
```

4. INLINE EXPORT VIA EXPORTS SHORTHAND

```
// math.js
exports.add = function(a, b) { return a + b; }
exports.subtract = function(a, b) { return a - b; }
exports.multiply = function(a, b) { return a * b; }
```

How require() Works Internally

1 Built-in Module Check

Kya ye Node.js ka built-in module hai? (http, fs, path, url) → Agar haan, Node.js installation se load karo.

2 Relative Path Check

Kya path `./` ya `../` se shuru hota hai? → Agar haan, us exact file path se load karo.

3 node_modules Lookup

Agar built-in nahi hai aur relative path nahi hai → `node_modules` folder mein search karo.

4 Execute & Cache

File ko completely run karo. `module.exports` mein jo bhi hai wo return karo. Agli baar same file require karo toh cached result instant return hoga.

PERFORMANCE

require() caching bahut powerful hai. Agar aap `require('./config')` 50 files mein karte ho, toh pehli baar file execute hogi, baaki 49 baar cached object return hoga. Isliye heavy initialization (DB connection) wale modules mein caching ka fayda uthaya jaata hai.

The Critical Trap — exports vs module.exports

IMPORTANT

exports sirf module.exports ka reference hai. Agar aap `module.exports = {}` se naya object assign kar doge, toh `exports` purana reference point karta rahega aur Node.js `module.exports` return karega — aapka `exports` wala data lost ho jaayega.

```
// ❌ WRONG — mulNumber silently lost
exports.mulNumber = function() {}
module.exports = { add, subtract };

// Why it breaks:
// exports        →  { mulNumber: fn }
// module.exports  →  { add, subtract }   // new object, connection broken
```

```
// ✅ CORRECT — pick one style, stick to it

// Style 1 — collect at bottom
module.exports = { add, subtract, multiply };

// Style 2 — export inline one by one
exports.add = function() {}
exports.subtract = function() {}
exports.multiply = function() {}
```

INTERVIEW TIP

"exports aur module.exports mein difference kya hai?" — Ye Node.js ka sabse common interview question hai. Answer: `exports` ek local variable hai jo `module.exports` ko point karta hai. Jab aap `module.exports = newObj` karte ho, toh `exports` ab bhi purana object ko point karta hai. Node.js hamesha `module.exports` return karta hai.

What Crosses the Boundary

Sirf `module.exports` hi dusri file mein jaata hai. Baki sab private rehta hai.

```
// math.js                          app.js
// ──────────────────               ──────────────────
let counter = 0;
function add() {}
module.exports = add    ──────→      const add = [sirf ye milta hai]

// counter yahin rehta hai.
// Sirf add cross karta hai.
```

ES Modules (ESM) — import & export

Why ESM Exists

WHY ESM? JavaScript ke official standard mein pehle koi module system nahi tha. Node.js ne 2009 mein **CommonJS** invent kiya apne liye. Browser alag tareeke se kaam kar raha tha. Har jagah alag-alag system tha.

TC39 (JavaScript standards committee) ne 2015 (ES6) mein decide kiya ki ek **official module system** hona chahiye jo **browser aur Node.js dono mein kaam kare**. Wahi hai ES Modules.

> *ESM JavaScript ka native module system hai. CommonJS Node.js ka invention tha. Future mein ESM dominate karega — new projects mein ESM use karna best practice hai.*

How to Enable ESM in Node.js

Node.js by default CommonJS assume karta hai. ESM enable karne ke do tareeke:

```
// Method 1: package.json mein type set karo
{
    "type": "module"
}

// Method 2: File extension .mjs use karo
math.mjs
app.mjs
```

Named Export

WHY named exports? Kyunki ek file se multiple cheezein share karni ho toh har cheez ka apna naam hona chahiye. Import karne waala exactly wo naam use karta hai.

```
// math.js
export function add(a, b) {
    return a + b;
}

export function subtract(a, b) {
    return a - b;
}
```

```
// app.js — exact name se import
import { add, subtract } from './math.js';

add(2, 3);         // 5
subtract(5, 2);    // 3
```

Default Export

WHY default export? Kyunki har file ka ek **primary purpose** hota hai. Wo main cheez default export hoti hai. Baaki supporting functions named exports hain.

```
// math.js
export default function add(a, b) {
    return a + b;
}
```

```
// app.js — no curly braces for default
import add from './math.js';

add(2, 3);    // 5
```

EDGE CASE

Ek file mein sirf ek default export allowed hai. Do default exports karoge toh syntax error aayega. Kyunki jab aap `import xyz from './file'` karte ho, toh naam specify nahi hota — ambiguity create hoti hai. **One file. One default. No exceptions.**

Default + Named Together

```
// math.js
export default function add(a, b) {
    return a + b;
}

export function subtract(a, b) {
    return a - b;
}

export function multiply(a, b) {
    return a * b;
}

export const PI = 3.14;
```

```
// app.js
import add, { subtract, multiply, PI } from './math.js';

// Default pehle aata hai, named curly braces mein.
// Default = file ka primary purpose
// Named = supporting utilities
```

The Critical Rule — import Must Be at Top

```
// ✅ Valid — top level
import { add } from './math.js';

// ❌ Invalid — condition ke andar nahi
if (something) {
    import { add } from './math.js';
}
```

INTUITION

Ye restriction actually performance ka reason hai. Kyunki imports hamesha top par hain, Node.js file run karne se pehle hi pure file ko scan kar leta hai, saare imports ek saath parallel mein load kar leta hai. Agar imports dynamic hote (condition ke andar), toh parallel loading impossible ho jaati.

require Does Not Exist in ESM

```
// package.json mein "type": "module" daalne ke baad:

const http = require('http');     // ❌ ReferenceError: require is not
defined

import http from 'node:http';     // ✅ correct — note the 'node:' prefix
```

BEST PRACTICE

ESM mein built-in modules ke liye `node:` prefix use karo — `import http from 'node:http'` instead of `'http'`. Ye explicitly batata hai ki ye Node.js ka built-in module hai, aur future-proof bhi hai.

CommonJS vs ESM — Head to Head

Feature	CommonJS	ESM
Syntax	`require()` / `module.exports`	`import` / `export`
File Extension	`.js` (default)	`.js` (with type:module) / `.mjs`
Import Location	Anywhere in file	Must be at top
Loading	Synchronous, one by one	Asynchronous, parallel
Tree Shaking	Not supported	Supported natively
Static Analysis	Hard (dynamic requires)	Easy (imports are static)
Browser Support	Needs bundler	Native support
Default in Node.js	Yes (until now)	Future default

INTERVIEW TIP

"Tree shaking kya hota hai?" — ESM mein bundler (Webpack, Rollup) build time par analyze kar sakta hai ki kaunse imports actually use ho rahe hain. Unused exports ko final bundle se hata diya jaata hai. CommonJS mein `require()` dynamic hai, isliye bundler statically analyze nahi kar sakta — tree shaking fail ho jaata hai.

Key Takeaways

Node.js Module Systems

- **CommonJS:** `require()` + `module.exports` — Node.js default, synchronous
- **ESM:** `import` + `export` — JavaScript standard, parallel loading, tree shaking
- `exports` sirf `module.exports` ka reference hai — dono mix mat karo
- Named exports unlimited hain; default export exactly one per file
- ESM mein imports hamesha top par hone chahiye — ye restriction parallel loading enable karti hai
- New projects mein ESM prefer karo — future-proof aur browser-compatible

TAKEAWAY

Module system bina samajhe aap scalable code nahi likh sakte. CommonJS abhi bhi widely used hai, lekin ESM future hai. Dono samajhna zaroori hai — legacy code mein CommonJS milega, new projects mein ESM. ESM prefer karo — tree shaking, parallel loading, aur browser compatibility ke liye.
